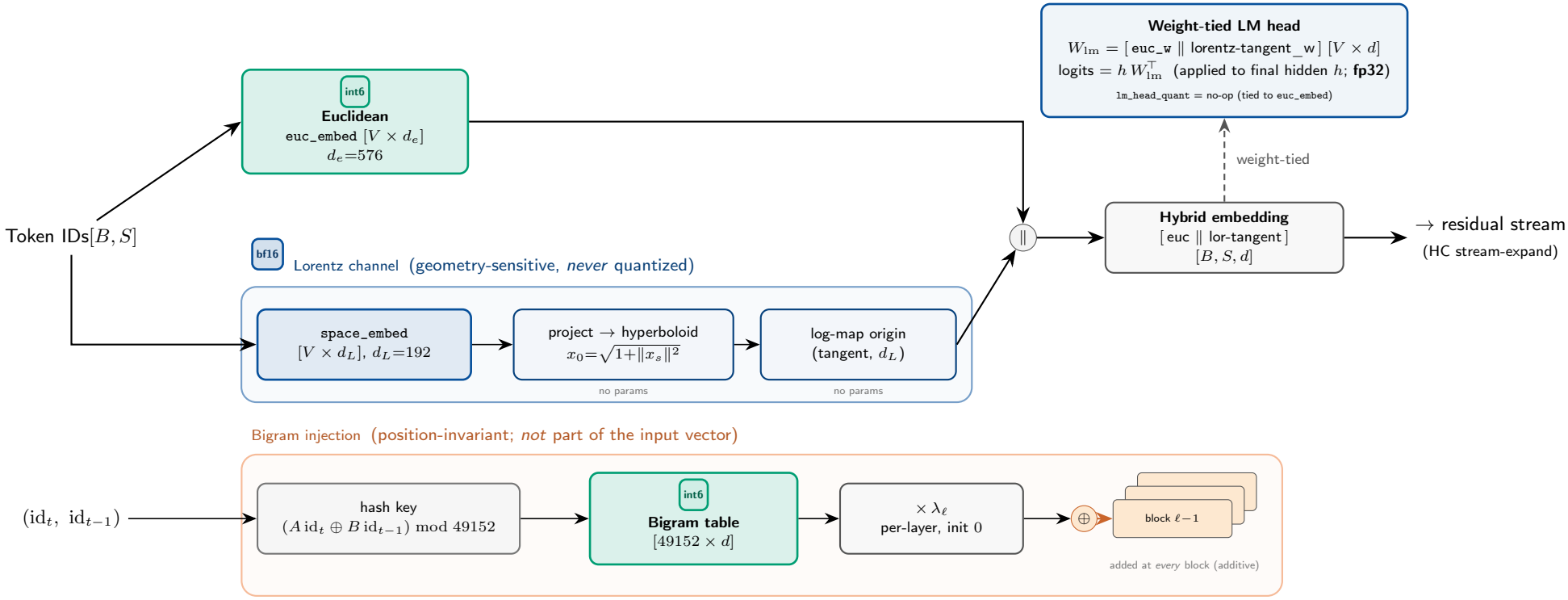
MORPH Embeddings — Hybrid Input + Bigram Injection

a hybrid Euclidean ‖ Lorentz input vector (computed once) plus a position-invariant bigram signal injected per layer; the LM head is weight-tied to the Euclidean + Lorentz-tangent weights



Precision:

int6 euclidean + bigram tables (per-row int-N QAT) **bf16** Lorentz `space_embed` (geometry-sensitive, *always*) + all geometry ops + fp32 logits

Weight-tying: the LM head has *no* separate weight — W_{lm} *is* $[\,\texttt{euc\_w} \,\|\, \text{lorentz-tangent\_w}\,]$, so quantizing `euc_embed` already moves the head; `lm_head_quant` is a no-op at the weight level. Logits are cast **fp32** before the chunked softmax (`fused_ce.py`).

Notation: V vocab, d model dim, $d_e{+}d_L{=}d$ (`lorentz_fraction`$=0.25 \Rightarrow 576{+}192$ at $d{=}768$), λ_ℓ per-layer bigram gate (init 0, learned). Default `embed_quant="off"` = bit-identical bf16; int6 shown is the *deployment target*.